Computer Networks 194 (2021) 108147

Contents lists available at ScienceDirect

Computer Networks

journal homepage: www.elsevier.com/locate/comnet

# Early detection of link failures through the modeling of the hardware deterioration process

Marco Polverini [a], Juan Luis Herrera [b], Pierpaolo Salvo [c], Jaime Galán-Jiménez [b,d,*]

[a] DIET Department, University of Rome "Sapienza", Rome, Italy
[b] Department of Computer Systems and Telematics Engineering, University of Extremadura, Cáceres, Spain
[c] Fondazione Ugo Bordoni, Viale del Policlinico 147, 00161, Rome, Italy
[d] Instituto de Investigación en Tecnologías Informáticas Aplicadas de Extremadura, Spain

ARTICLE INFO

*Keywords:*
Link failures
Hardware deterioration
Bidirectional Forwarding Detection
Fast Re-Route
QoS

ABSTRACT

Network failures can significantly degrade the network performance in several ways, ranging from experiencing unacceptable delays (e.g., in video conference applications) to leading to significant drops in sales for companies (e.g., airlines tickets purchases). In this paper, we focus on network failures caused by links that are prone to fail due to their advanced stage of lifetime. In particular, the formalization of the Hardware Deterioration Failure Mode (HDFM) for network line cards is provided to evaluate the penalty for network operators of handling HDFM failing links using classical methods based on Bidirectional Forwarding Detection and Fast Re-Route mechanisms. An algorithm, named Hardware Deterioration Detection (HDD), is proposed to detect if links are experiencing failures as well as to provide an estimation of their remaining lifetime. The performance evaluation over data generated by our provided network failure simulator shows that HDD is able to correctly classify links as failing or not failing, regardless of the type of failure considered. Moreover, overall descriptions for failure models in which HDD is able to produce high quality estimations of the remaining lifetime have been identified, with a trade-off between the hardware deterioration time dependence and the relationship between lifetime and packet loss probability.

## 1. Introduction

Nowadays, a high percentage of applications require an "always-on" network connectivity with optimal values for Quality of Service (QoS) parameters (e.g., low delay and jitter, high throughput, etc.) [2]. This availability is directly related to the performance of the network infrastructure, whose elements can be affected by different types of failures.

Network failures are negative events that affect the correct behavior of the network, leading to QoS degradation or service disruptions, which can severely impact on business revenues [3]. The term *network failure* includes a wide set of events, ranging from *logical failures*, such as a destination prefix not included in the routing table of a router or a routing misconfiguration leading to the creation of a loop, up to *physical failures*, such as optical equipment failure, fiber cuts, etc.

In [1], physical network failures are characterized based on planned and unplanned. While the firsts are related to maintenance operations that can be easily managed through a prior traffic detour using protection schemes, the latter ones are stochastic events that are hard to detect and need to be quickly handled in order to mitigate their impact on the network performance. Unplanned failures can be further divided into *shared* and *single* link failures. Shared link failures are either due to router related service disruptions, that can cause multiple links to simultaneously become unavailable or, e.g., due to a failure of an optical component that is handling multiple Layer-3 (L3) connections. On the other hand, single link failures are events that determine the change of the status on a single L3 connection, and are related to faults that occur on a single line card.[1] Moreover, in [1] it is also found that links can be divided into two sets depending on how often they fail, i.e., Low and High Failure Links (LFLs and HFLs). LFLs failures are caused by sporadic events whose reason is most of the times unknown: these events are rare and the duration of a single failure (time to repair) can be quite long (several hundreds of seconds), since their restoration requires the intervention of the network operator. Failures events involving LFLs are difficult to predict, due to the fact that they are somehow unexpected. On the contrary, failures involving HFLs represent a large fraction of the total number of failure events at the

* Corresponding author at: Department of Computer Systems and Telematics Engineering, University of Extremadura, Cáceres, Spain.
*E-mail addresses:* marco.polverini@uniroma1.it (M. Polverini), jlherrerag@unex.es (J.L. Herrera), psalvo@fub.it (P. Salvo), jaime@unex.es (J. Galán-Jiménez).

[1] In [1] it is shown that the largest percentage of single link failures is due to L3 components.

https://doi.org/10.1016/j.comnet.2021.108147
Received 3 November 2020; Received in revised form 14 March 2021; Accepted 28 April 2021
Available online 3 May 2021
1389-1286/© 2021 Elsevier B.V. All rights reserved.

network level [1], though concentrated over a restrained number of links, and their time to repair is generally of the order of few seconds. Furthermore, since HFLs are highly prone to fail, their status changes are more likely to be predicted: knowing that a given link is of type HFL, it is likely that it will fail in the near future so that the operator can act in advance. Failure events regarding HFLs are mainly due to the advanced stage of their lifetime (hardware deterioration), which can cause sudden status changes in some of their components.

An early detection of the presence of LFLs in the network can help the operator to take actions on the routing strategies (i.e., Fast-Re-Route (FRR)) in advance in order to reduce the potential impact on the experienced QoS caused by the failing LFL. However, the situation changes when acting upon detecting HFLs. Since their correct operation is intermittent due to hardware deterioration, i.e., time periods working properly followed by others in which a high percentage in the packet loss is experienced, the application of continuous FRR operations could be even worse than continuing with the current routing configuration [4].

For this reason, in this work we propose a strategy to manage failure events due to the hardware deterioration in HFLs. Based on a supervised Machine Learning (ML) approach, the Hardware Deterioration Detection (HDD) algorithm aims at detecting HFLs and provide an estimation of their remaining lifetime. With this information, the operator is able to decide the action to take on the failing link. In order to do that, HDD exploits an important characteristic which is typical of hardware deterioration [5], i.e., the fact that this process is non stationary. To better capture the aspect of the failure events affecting the HLFs, we introduce the concept of Hardware Deterioration Failure Mode (HDFM) on network line cards. In particular, after a link starts failing, a period of time in HDFM is associated to such link, which finally stops working. During this period, the link status changes over time, continuously passing from OFF to ON and vice versa.

Due to the lack of real data sets of line cards lifetime, a network failure simulator is provided, which is used to generate the input data set to test the validity of the proposed solution. Despite the use of real data would allow to get more reliable evaluation, by means of the proposed network failure simulator we are able to stress the HDD algorithm, by considering different pessimistic scenarios (e.g., stationary functions for HDFM). By applying the considered methodology, we have found that also in case of stationary HDFM (or with a negligible time dependence), HDD is still able to correctly classify links to as HFLs or LFLs.

The present work extends our previous contribution [6], by including the following new aspects:

- the formalization of the Hardware Deterioration Failure Mode (HDFM) for network line cards;
- a deep discussion on the negative impact of handling HDFM related failures using classical approaches;
- a novel network simulator for the generation of failure events on HFLs, in which the functions used to model the hardware deterioration process are able to generate a series of failure events that follow the same statistical distribution as the one reported in [1], which was validated over real data;
- an evaluation of the QoS degradation due to the use of classical failure recovery approaches when those are used to react to HDFM failures;
- an extensive performance evaluation;

The rest of the paper is organized as follows: Section 2 provides a description of failure events happening on HFLs. The definition and formalization of HDFM is presented in Section 3. In Section 4, the proposed HDD algorithm to detect failing links and to predict their remaining lifetime is described. Section 5 presents the network failure simulator created to test HDD. Performance evaluation is reported in Section 6, while a set of lessons learned from our experience are described in Section 7. Section 8 reviews the related work on failure detection and prediction in communication networks. Finally, conclusion and future works are presented in Section 9.

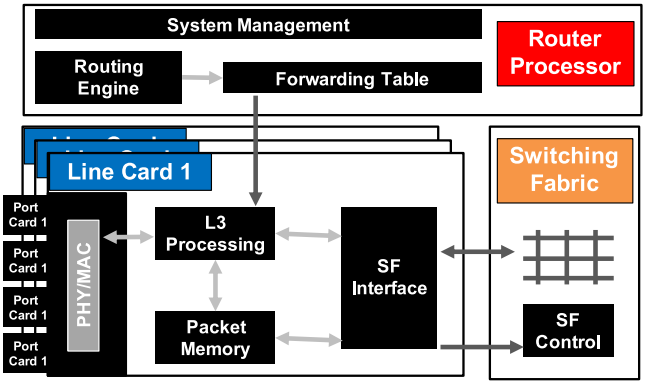



**Fig. 1.** Router architecture.

## 2. Understanding the nature of failures in HFLs

As defined in [1], HFLs are links that suffer of intermittent and recurring faults affecting their components, due to the advanced status of their lifetime. These failure events cannot be related to any of the classical types of failures happening in a network infrastructure. Then, in the next we describe the nature of such negative events.

As first, it is important to have a clear view of the architecture of a network device. The general model of the main functional blocks of a router is shown in Fig. 1 [7]. In the chassis[2] the following components are hosted in the form of different electronic circuit boards: (i) the *router processor*, which performs control plane related functionality, (ii) the *switching fabric*, which is in charge of switching incoming packets toward the output port, and (iii) the line cards. With reference to the line cards, they perform operations that go from the network layer up to the physical layer. In particular, the main functions hosted in a line card are the *switching fabric interface* that realizes the connection of the line card with the switching fabric, the *forwarding engine* that performs packet processing (queuing, routing table look-up, etc.), and a set of ports that realize the TX/RX operations. All these functions are implemented in a single electronic circuit board.

In [8] the term *failure mechanism* is used to refer to the degrading effect of the time over the hardware components. It represents the set of chemical, physical, or material process that deteriorates the card, leading to a worsening of the performance and of the proper operation. Depending on the specific component that is deteriorated by these processes, the hardware can fail following different patterns, referred to as *failure mode* in [8]. It can suddenly stop working, or it can slowly experience a loss of performance, alternating ON and OFF periods, until it reaches the non operational status. Data reported in [1,9] shows that network line cards exhibit a failure mode of this second type, i.e., the deterioration process lasts over a long time horizon, where the card intermittently experiences status changes.

Unfortunately, due to the fact that the hardware autonomously recovers from the OFF status and after a short period, it is difficult to determine the source of the fault event. This phenomenon is referred in the literature [10] to as No-Fault-Found (NFF). Anyway, the temporary outage experienced by the line card can be observed from outside, since it has a negative impact on the amount of packet loss. This suggests that through the monitoring of this parameter it is possible to diagnose the aging level of the hardware composing the line card. Statistics related to the packet loss are collected at the IP link level. Then, the way IP links are mapped on the line card plays a crucial role in the correct interpretation of these data.

Two different mappings can occur in a network: (i) *one to one*, where all the ports of a line card are associated with a single IP link, and (ii)

[2] It also hosts the cooling system and the power supply.

*ports bundle*, in which only a subset of ports of a line card are mapped with an IP link (in this case, multiple IP links can be related with a single line card). Moreover, with reference to the router architecture presented in Fig. 1, faults to HFLs can be divided into two categories depending on the functionality of the line card that is experiencing the outage: (i) *shared*, i.e., the failing component is used by all the IP links mapped with the line card (e.g., the packet processing), and (ii) *single*, regarding outages involving components associated to a single IP link (e.g., a single port). In the next we assume that the *one to one* mapping is used and that only *shared* failures are caused by the hardware deterioration process.

## 3. Hardware deterioration failure mode

The deterioration of line cards that are in advanced stage of their lifetime can cause temporary outages on some of their hardware components [1], that lead to sudden status changes in HFLs. In [10], the authors introduce the concept of intermittent failures. A discussion about the requirements for observations (e.g., no-fault-found ones) to be considered as failures is provided, as well as guidelines for the assessment of intermittent failures.

We define the *Hardware Deterioration Failure Mode* (HDFM) as the continuous change status of HFLs. The HDFM impacts the performance of network links in such a way that outages in their physical components can lead to a sudden increase of the packet loss, reaching the value of 100% for a small period of time. Then, after the outage period, this value is decreased down to 0%.

In the context of an Autonomous System, link failures are generally detected using an *hello based* protocol [11], where two connected line cards monitor their status by exchanging periodic hello messages. If a line card does not receive hello messages during a period of time longer than a threshold (named *dead interval*), then it assumes that the link is down and Link State Advertisements (LSAs) are flooded to notify this event at the network level. If a link is experiencing a HDFM and temporary outages are caused at the same time when hello messages from the other end are expected, a link status change can be experienced.

An example of such a phenomenon is reported in Fig. 2, where red arrows represent the time instants when hello messages are expected and the blue areas indicate the outage periods for the line card (due to the HDFM) where the instantaneous packet loss reaches 100%. Therefore, if a red arrow falls into a blue area, then the related hello message is lost. In the considered example, three consecutive hello messages are lost. They are sent every *hello interval* seconds and none of them successfully arrive. Therefore, after the duration of the *dead interval*, the link is detected as down. Next, a new hello packet is received (even an additional outage period is previously experienced) and the link is detected as restored. As shown in [1], this situation is mostly experienced just after the duration of a hello interval (as in the example shown in Fig. 2), e.g., 15 s if the default setting is considered [12]. This is the reason why the time to repair for failures involving HFLs is generally of the order of few seconds.

With the aim of providing a description of the behavior of a link affected by the HDFM, in the next we introduce the *hardware deterioration function* ($\Phi(t)$). As shown in Fig. 3, the life of a line card experiencing an HDFM can be divided into three main phases: (i) working, (ii) failing, and (iii) not operational. Initially, the line card is in the *working* status, meaning that it behaves normally and packet loss is mostly due to congestion. Then, due to the advanced stage of its life, the line card enters the *failing* status, where some of its components can experience temporary outages, as previously explained. Consequently, the cause of packet loss can be either due to the failing status and/or to congestion. The last phase of the life of a line card starts when it passes from *failing* to the *not operational* status. In this case, the line card is not able to process packets, and the packet loss stabilizes at its maximum,

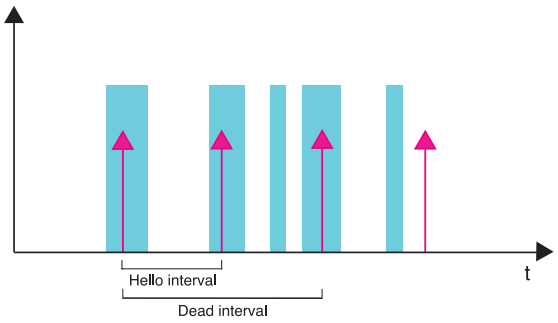



**Fig. 2.** An example showing how a line card experiencing an HDFM is detected as DOWN. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

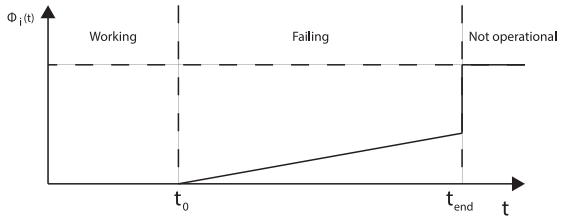



**Fig. 3.** An example of *hardware deterioration function*.

i.e., 100%. The aim of the *hardware deterioration function* is therefore to model the behavior of the line card when it is in the *failing* status.

The *hardware deterioration function* is defined as the average probability that the line card $i$ is in OFF status over time, while it is in the *failing* status. Formally, referring to $t_0$ as the time instant when the line card moves from the *working* status to the *failing* one, with $t_{\text{end}}$ as the time instant when the line card enters in the *not operational* status, the average probability of the line card $i$ to be in OFF status ($P_{\text{OFF}}(t)$)[3] is expressed by Eq. (1):

$$P_{\text{OFF}}(t) = \begin{cases} 0 & t \le t_0 \\ \Phi(t) & t_0 \le t < t_{\text{end}} \\ 1 & t \ge t_{\text{end}} \end{cases} \tag{1}$$

A simple example of the hardware deterioration function is reported in Fig. 3. In this case, a simple linear relation is shown, but in general it can follow any function [6]. The only assumption that we make is that the link hardware deterioration process experienced by a line card in HDFM is *non-stationary*, meaning that $\Phi(t)$ changes over time.[4] Furthermore, at point $t = t_{\text{end}}$ the function $\Phi(t)$ can present a discontinuity ($\Phi(t_{\text{end}} - \epsilon) \neq \Phi(t_{\text{end}} + \epsilon)$, with $\epsilon \to 0$).

Clearly, the hardware deterioration function is strongly related to the packet loss probability at a given time $t$ due to the HDFM. In fact, if the line card is in OFF status at time $t^*$ (which occurs with probability $\Phi(t^*)$), then the packet loss probability is equal to 1. Thus, the hardware deterioration function provides an indication of the actual QoS degradation due to the failing status that the line card is experiencing. Unfortunately, the hardware deterioration function is in general unknown, neither is known whether a line card is experiencing the HDFM. In the following, we introduce a ML-based approach to estimate one of the parameters characterizing the hardware deterioration function, i.e., the remaining lifetime ($\delta_T$) of the line card ($\delta_T = t_{\text{end}} - t_0$).

[3] Note that the $P_{\text{OFF}}$ probability might depend on several parameters. Anyway, in this work we are interested only on its dependence with respect to the time.

[4] This assumption is in line with deterioration processes regarding different physical systems [5].

Furthermore, we have developed a tool in order to detect whether a line card is in HDFM status based on its estimated lifetime.

### 3.1. The importance of diagnosing the HDFM of a line card

In this subsection we provide a discussion about the importance of correctly diagnosing if a line card is experiencing the HDFM, as well as posing the basis for a more appropriate way of handling this type of failures. Then, in the next section we present HDD, a framework that allows to detect line cards that are working in HDFM status.

The first step toward the definition of a strategy to handle failures due to HDFM is to dive into the differences between events affecting HFLs and LFLs. With reference to the example shown in Fig. 2, three main outcomes can be derived: (i) the line card is up for most of the time, but (ii) due to temporary outages happening concurrently with the expected reception of hello messages, the link is seen as down, and (iii) also during the time period when the link is seen as up, there is a fraction of time where the line card is in OFF status. The previous observations highlight an important difference between failures experienced by HFLs and the ones related to LFLs. While the first ones exhibit continuous and intermittent ON/OFF behavior, the latter group spends a long period of time in OFF status.

Due to the different evolution of the failure modes, these events should be addressed in different ways. On the contrary, in current networks the type of failure mode is not taken into account in the choice of the mechanism to be used to react to a failure event, and generally FRR strategies [13] are applied: when the failure is detected, the traffic is diverted over a (possibly pre-computed) alternative path, and once the link is restored, the traffic is re-routed over the original path. It is clear that FRR is the right approach to react to a failure involving a LFL, since in this case the link can be down for a long period of time, but after restoration, it works normally. Contrarily, link status changes considering a line card in HDFM are more frequent, determining a high number of re-routing operations, that can negatively affect the network stability [4]. Furthermore, whenever a HFL changes its status from OFF to ON, this does not imply that it will start working normally, since it is still in HDFM. Consequently, when the traffic is moved back on the original path that now it is seen as restored, a fraction of it will be lost due to the HDFM.

The correct way of handling failures due to the HDFM is then to dismiss the old line card since it is not more reliable. Obviously, due to the fact that this strategy has a considerable impact in terms of Capital Expenditure (CAPEX), it is fundamental to have a reliable diagnosis of the fact that the line card is actually in HDFM. Moreover, in some cases the line card could be in an early stage of its hardware deterioration process, then the network operator might accept the QoS degradation derived by its usage, compared with the cost of a link replacement. For these reasons, in the next section we present HDD, a ML-based approach that allows to detect if a line card is in HDFM status and to estimate its remaining lifetime.

## 4. The Hardware Deterioration Detection framework

In this section, the proposed framework is described. Starting from the explanation of the network architecture in which the solution is intended to be applied, the description of how the data set is generated and the ML algorithm used to detect a failing line card as well as to predict its remaining life time are provided.

### 4.1. Network architecture

As previously introduced, the main goal of the proposed framework is to detect if a line card is experiencing the HDFM and to assess its remaining lifetime. Based on this information, the network operator can then decide how to react against this failure event. In order to do that, the proposed algorithm does not rely on classical detection techniques, such as BFD or LOS. Its aim is to detect if there is an anomalous behavior in the statistics related to the packet loss and link load.

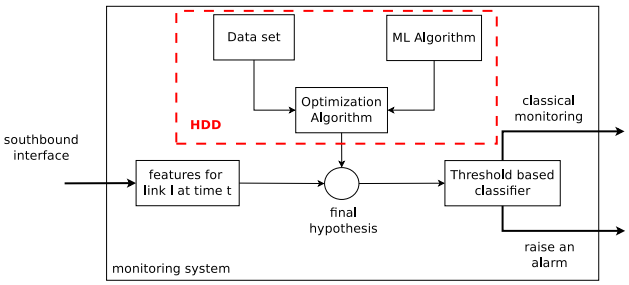



**Fig. 4.** Architecture of the monitoring system based on HDD algorithm.

The proposed framework, namely Hardware Deterioration Detection (HDD) is intended to be integrated in a centralized monitoring system, as depicted in Fig. 4. Firstly, the HDD algorithm is used to generate a final hypothesis, i.e., a function that mimics the mathematical model that regulates the remaining lifetime of network line cards. Then, the final hypothesis is used to predict the remaining lifetime of line cards, by providing it with the current set of features (which will be described later) as input. Features are periodically collected from the network devices by means of a southbound interface, which can be realized by different protocols (OpenFlow, SNMP, etc.). The signaling traffic generated by the data collection phase depends on several factor, as the amount of data to be collected from every line card, the number of line cards and the frequency at which this process is repeated. By the way, considering normal application context, the required capacity is negligible. No further functions need to be implemented, since simple OpenFlow OFP.STAT or equivalent messages can be used to get the required data, e.g., TX/RX traffic counters [14]. Finally, the output estimation is provided as input to a threshold based mechanism that classifies the line card as failing or not failing. The classification is performed as follows: if the estimated lifetime is below the threshold, then the link is declared as *failing*. In the next subsections, the main functional blocks reported in Fig. 4 are detailed.

### 4.2. Data set

The proposed HDD framework exploits a supervised ML approach, meaning that, in order to be properly trained, a data set containing labeled data is required. Each data sample $s = \{f_1, f_2, \ldots, f_i, \ldots, f_n; l\}$ in the data set, also referred to as *observation*, is composed of a *label* $l$, which represents the actual output to be predicted, and of a set of *features* $f_i$, i.e., gathered information used to predict the output.

Since HDD aims at estimating the remaining lifetime of a line card, this value is used as the label for each observation. The selection of the line card lifetime as the label is an important design choice made in HDD. In fact, with the aim of allowing the operator to decide whether to substitute an HDFM failing line card based on the possible QoS degradation, the estimation of the $P_{\text{OFF}}$ probability would be more helpful than the remaining lifetime (since it is directly related to performance loss). Moreover, the packet loss probability could also be considered as the label of the algorithm in order to try to predict the QoS of a line card. However, here we argue that $P_{\text{OFF}}$ and packet loss probabilities cannot be measured since the only available statistic that can be used for prediction, apart from link load, is the overall packet loss associated to a line card. This value can be the result of a wide set of potential situations, such as congestion, checksum errors, different errors reported by ICMP protocol (destination unreachable, TTL expired, etc.), hardware failures derived from different layers, etc. Therefore, the relationship between the packet loss and the HDFM is

not straightforward, since it is difficult to infer the specific contribution of the failure mode over the packet loss obtained value.

On the contrary, the remaining lifetime can be easily measured and provides a clear indication of the current status of the hardware under study, by collecting data as follows: (i) at each time slot, the network operator gathers the selected features from each line card and assigns a time stamp $t_i$ to this observation, (ii) then, once the line card passes from *failing* to *not operational* status at a given time slot, $t_{\text{end}}$, it comes back on each previously collected data related to the *not operational* link and sets for it a label given by $l = t_{\text{end}} - t_i$.

The data collection process is performed by the central monitoring system which also executes HDD. Network nodes are periodically queried, and the time between two consecutive queries is named time slot. The duration of a time slot is a parameter that can be selected by the network operator. It affects the performance of HDD in two ways. As first, since the aim of HDD is to estimate the remaining lifetime of line cards, the duration of the time slot imposes the sensitivity of the output. Then, the reduction of the duration of a time slot has a beneficial impact on the sensitivity of the proposed method. On the contrary, the duration of a time slot also determines the size of the data set, that is inversely proportional to this quantity. Consequently, there is a trade-off between the sensitivity of the output and the computational complexity of the training phase of HDD.

With regards to the set of features of the algorithm, these must be selected among those network parameters having the following two properties: (i) they must be measurable quantities, and (ii) they must be related to the failure event. With this in mind, in the HDD design, four features per line card are considered: $f_1$ is the link load at time slot $t$, $f_2$ is the packet loss at time slot $t$, $f_3$ is the link load at time slot $t-1$, and $f_4$ is the packet loss at time slot $t-1$. In particular, link load has been selected as feature since it is related to one of the strongest contributions to the line card packet loss, i.e., congestion. Moreover, packet loss has been selected as feature due to the fact that the HDFM failure affects this quantity.

Clearly, there are several other interesting features that can add more information and thus improve the accuracy of the proposed algorithm. Some examples are the packet arrival distribution, the buffer occupation, the line card CPU usage, the temperature, etc. Another example is the possibility to include the mapping of the IP topology over the physical network. This would allow to extend the validity HDD also to more complex scenarios, where *ports bundles* are used.

In the design of HDD we have decided to reduce the number of required features in order to keep the monitoring system as simple as possible, since the collection of the aforementioned parameters is challenging in practice. On the contrary, the selected set of features can be easily gathered through classical network monitoring interfaces. Anyway, in case the network operator has some means to obtain such values, then these can also be included in the set of features. Consequently, the considered data set structure can be seen as the minimum set of features that allow the system to obtain useful results for the network operator.

The general idea behind the selected features is that, looking at the link load level, HDD can roughly evaluate the amount of packet loss due to congestion and to filter it out from the statistic of the packet loss, so that the contribution of the HDFM is easier to detect. Furthermore, in order to consider the time dependence of the HDFM, each observation contains features related to two consecutive time instants.

### 4.3. ML and optimization algorithms

The HDD framework is modular and allows to apply different ML algorithms. In the next we consider a simple Linear Regression (LR) algorithm, consequently the final framework is referred to as Linear Regression-based Hardware Deterioration Detection (LR-HDD). Similar to Ordinary Least Squares regression (OLS), LR is an ML algorithm able to learn a mathematical function, known as *final hypothesis* or $h_\theta(x)$. In LR, $h_\theta(x)$ has the shape of a multidimensional line, which is the line that fits best the patterns of the data set. This is achieved by optimally minimizing a cost function known as $J(\theta)$, which expresses the difference between $h_\theta(x)$ and the data set in absolute terms. In the problem at hand, the $J(\theta)$ function used for training in our LR-HDD algorithm is shown in Eq. (2), that represents the half of the Mean Squared Error (MSE). Since the derivative of $J(\theta)$ needs to be assessed during training, the MSE metric is selected for simplicity. Its value is halved so that the multiplication by two appearing in the derivative is canceled out, and simplifies the calculations during the training process.

$$J(\theta) = \frac{1}{2m}\sum_{i=1}^{m}(h_\theta(s^{(i)}) - l^{(i)})^2 \tag{2}$$

In Eq. (2), $m$ represents the number of training examples; $h_\theta(x)$ is the output of the ML system given the input $x$ (also known as *value of the hypothesis function on* $x$); $s^{(i)}$ is a vector with the values for the features for the ith training example and $l^{(i)}$ is the label for the ith training example.

In this algorithm, $h_\theta(x)$ is a line that outputs a value for certain values of the features. As Section 4.2 explains, LR-HDD considers four features. Therefore, the model can be considered as a *multiple regression* model, since multiple features are taken into account [15]. If we have an example $s = \{f_1, f_2, f_3, f_4\}$ we can write $h_\theta(x)$ as shown on Eq. (3).

$$h_\theta(s) = \theta_0 + \theta_1 f_1 + \theta_2 f_2 + \theta_3 f_3 + \theta_4 f_4 \tag{3}$$

During the learning phase, the system tries to minimize $J(\theta)$ by using the training set to assess the different $\theta_i$ values through optimization, so that $h_\theta(x)$ fits the data as much as possible (i.e. $h_\theta(s^{(i)}) \approx l^{(i)}$). Since many different optimization algorithms are able to minimize this function, a modification of the L-BFGS [16] algorithm has been used.

During the performance evaluation, different ML models have been considered, such as Neural Networks (NN) and Support Vector Machine (SVM). Anyway, due to the fact that the considered data set has few features, and the training and test sets used for simulation are synthetically generated, the availability of more flexible models do not allow for important performance improvements. On the contrary, a loss of accuracy has been experienced during the evaluation of NN and SVM in some cases, while the results provided by LR have been the ones presenting the best outcomes.

Anyway, due to the modular structure of the proposed framework, more enhanced ML models can be considered in case of the availability of a larger set of features or a real data set.

## 5. HFDM simulator

In this section we provide the insights of the HFDM simulator that we have developed to generate the data set used to test LR-HDD. In fact, due to the lack of a real data set containing the information required by LR-HDD, we have generated a synthetic one. The main components of the HDFM simulator are: (i) a network environment composed of a topology, a set of links capacity and a series of traffic demands, (ii) a process to model packet loss caused by congestion, (iii) a method to select the line cards affected by the HDFM, (iv) a set of *hardware deterioration functions* to simulate the failure of the failing line cards.

### 5.1. Network environment setup

The HDFM simulator requires as input a network topology represented by a directed graph, $\mathcal{G}(\mathcal{N}, \mathcal{L})$, where $\mathcal{N}$ is the set of $N$ nodes and $\mathcal{L}$ is the set of $L$ links. Each link $l \in \mathcal{N}$ is associated to a single line card and has a capacity $c_l$. Furthermore, a set of Traffic Matrices $\mathcal{T}$ is required to model the traffic demands routed over the network. We assume that the time axis is divided into time slots, and we refer to as $T(t)$ to the traffic matrix in the time slot $t$. Each element $T(t)_{s,d}$ represents the aggregated amount of traffic entering the network from

node $s$ and leaving it through the node $d$ at time $t$. In the simulator we assume that each Ingress-Egress flow $T(t)_{s,d}$ is composed by a set of Origin–Destination (OD) flows, each one being represented by a Constant Bit Rate (CBR) source, having 1 Mbps of throughput and considering all the packets having the same length $L_{\text{PKT}}$.

In each time slot, the HDFM simulator calculates the IGP routing according to a shortest path rule (using the Dijkstra algorithm), and then computes the aggregated traffic carried by each link $l$, as the summation of the OD flows routed over the considered link.

### 5.2. Link congestion

In order to simulate the packet loss due to congestion events in the HDFM simulator the two following steps are executed: (i) each link is assigned with a buffer length ($b_l$), and (ii) the probability $Q(b_l)$ to have more than $b_l$ packets on link $l$ is evaluated. With respect to the second point, the following Eq. (4), extracted from [17], is considered:

$$Q(b_l) = \sum_{n=b_l+1}^{F_l} \binom{F_l}{n} \left(\frac{n-b_l}{D_l}\right)^n \left(1-\frac{n-b_l}{D_l}\right)^{F_l-n} \frac{D_l - F_l + b_l}{D_l - n + b_l} \tag{4}$$

where the term $F_l$ is the number of OD flows routed over the link $l$, $D_l$ represents the number of packets that the link $l$ can transmit during a single time period between two consecutive packets of the considered CBR source, and it is calculated as the ratio between the link capacity ($c_l$) and the rate of a single CBR flow (1 Mbps).

The obtained value of $Q(b_l)$ is used as packet loss probability due to congestion events during the current time slot. In particular, for each packet a random number is generated (according to a uniform distribution) and, if its value is lower than the value of $Q(b_l)$, then it is dropped. At the end of each time slot, before the processing of the next traffic matrix starts, a set of aggregated information for the current time slot is returned as output. Specifically, the HDFM simulator returns, for each link, the number of dropped packets due to congestion and the link utilization. Note that this information is available also in real network interfaces [14].

### 5.3. Failing line card selection

According to the analysis on real data on network failures conducted in [1], HFLs involve around the 2.5% of the line cards. Then, in the simulator (at the beginning of a new simulation) a number of line cards equal to 2.5% of the total, is randomly selected and declared as affected by the HDFM. For each of them, the time at which the failure starts is randomly chosen (with a uniform distribution in $(0, |\mathcal{T}|)$. All the remaining line cards are classified as LFLs and can fail with a probability $P_{\text{LFL}}$. When a line card experiences such a type of failure, it directly passes from *working* to *not operational* status, since failures on LFLs are not related to the hardware deterioration process.

### 5.4. Hardware deterioration function

Regarding the hardware deterioration function, in the simulator we have considered a simple linear trend, as the one shown in Fig. 3. There are two main motivations for this choice: (i) as first, in [6] we have tested the performance of LR-HDD considering different types of functions to model $\Phi(t)$, and we have found that the estimation error in the lifetime is almost identical, and (ii) by imposing a limitation on the slope of the linear models, we can control the time dependence of the deterioration process, meaning that we can evaluate worst case scenarios. Furthermore, we have experimentally found that considering a linear function to model $\Phi(t)$, it is possible to generate a series of failure events having the same statistical distribution as the one reported in [1], which has been validated over real data. As an example, in Fig. 5 we show the normalized number of failure events per link, considering only links affected by the HDFM. In this analysis, failure events are detected using the hello based protocol introduced in Section 3 and exploited in [1], where the data set used is represented by ISIS messages. We remind here that our proposed solution is not based on classical detection techniques, but on the information collected from network devices related to packet loss and link load throughout time. The dashed line reported in Fig. 5 represents the power law (a straight line considering log–log axes) that best fits the points. According to what is found in [1], the normalized number ($nn(l)$) of failure events[5] on link $l$ generated considering $\Phi(t)$ modeled with linear functions follows the power law $nn(l) \propto l^{-0.73}$.

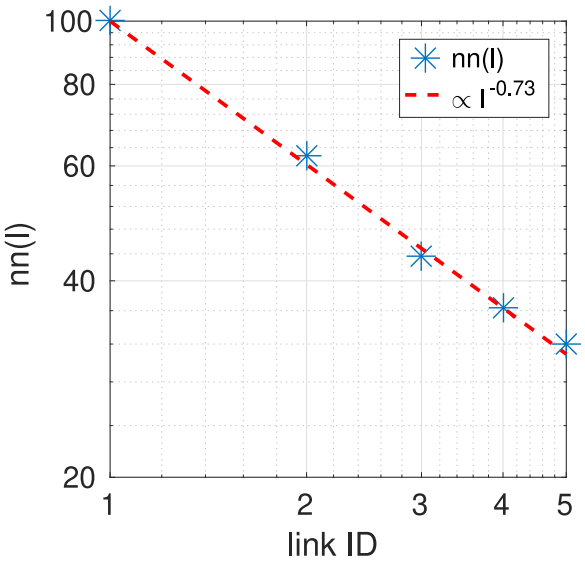



**Fig. 5.** Normalized number of failure events. As it can be seen, the points follows a power law that is proportional to $l^{0.73}$.

The considered linear trend is represented by the straight line that passes through the points $P_1 = (0, \phi - 0.5\lambda)$ and $P_2 = (\delta_T, \phi + 0.5\lambda)$, where $\delta_T$ is the duration of the failing phase, $\phi$ represents the average probability for the line card to be in OFF status during the failing phase, and $\lambda$ provides a measure of the variability of the $P_{OFF}$ probability over time.[6] In order to evaluate the performance of LR-HDD over failures described by different hardware deterioration functions, we have divided the failure modes into different groups. In our simulator we have considered the values reported in Table 1 (the analysis of Fig. 5 is referred to models in group 1 of Table 1).

At the beginning of a simulation, for each failing link, the HDFM simulator randomly selects a failure mode and a time slot indicating when the link enters the failing status. Then, similarly to the case of packet loss due to link congestion, for each packet a random variable is selected and compared with the packet loss due to the HDFM. If the random variable is lower than this quantity, the packet is dropped.

### 5.5. Using the HDFM simulator to generate a data set

Next we provide a brief overview of the overall process that is followed to generate the data set to train and test the LR-HDD framework. As previously explained, the data set $D$ is composed of a collection of observations, $d$. Each observation includes four features and a label. The features are the link congestion in current and previous time slots, and the percentage of packet lost in current and previous time slots.

[5] We have also verified the validity of the other considered distributions for HFLs available in [1], i.e., CDF of the Time To Repair and Mean Time Between Failures.

[6] When $\lambda$ is equal to 0, the *hardware deterioration function* is a constant, meaning that the deterioration process is stationary.

**Table 1**
Different hardware deterioration functions, divided into 4 groups.

| Group | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | | | 2 | | | 3 | | | 4 | | |
| $\delta_T$ | $\phi$ | $\lambda$ | $\delta_T$ | $\phi$ | $\lambda$ | $\delta_T$ | $\phi$ | $\lambda$ | $\delta_T$ | $\phi$ | $\lambda$ |
| 96 | 0.40 | $10^{-2}$ | 960 | 0.40 | $10^{-3}$ | 480 | 0.20 | 0.40 | 288 | 0.25 | 0 |
| 144 | 0.28 | $10^{-2}$ | 1440 | 0.28 | $10^{-3}$ | 720 | 0.15 | 0.30 | 576 | 0.16 | 0 |
| 192 | 0.22 | $10^{-2}$ | 1920 | 0.22 | $10^{-3}$ | 960 | 0.12 | 0.24 | 864 | 0.13 | 0 |
| 240 | 0.19 | $10^{-2}$ | 2400 | 0.19 | $10^{-3}$ | 1200 | 0.10 | 0.20 | 1152 | 0.11 | 0 |
| 288 | 0.17 | $10^{-2}$ | 2880 | 0.17 | $10^{-3}$ | 1440 | 0.085 | 0.17 | 1440 | 0.09 | 0 |

The label is represented by the remaining lifetime. If the considered line card is affected by the HDFM and the failure has started, then the label is calculated as the difference between the time slot when the line card will stop working minus the current time slot. In all the other cases, the label will be assigned to a default (high) value. This process is executed at each time slot.

Each run of the HDFM simulator is able to produce a large number of observations. Specifically, referring to $|\mathcal{T}|$ as the total number of time slots, a single execution of the simulator provides $|\mathcal{T}| \cdot L$ observations. Anyway, in the creation of the data set, all the observations that are related to non failing links are removed. Clearly, in order to have a good statistic and to use all the considered failure modes, the HDFM simulator has to be launched many times. The final data set is obtained as the union of the observations produced by each single run.

## 6. Performance evaluation

In this section we provide an extensive performance evaluation of the proposed LR-HDD framework. To do that, we generate a data set by means of the HDFM simulator presented in Section 5. The section is organized as follows: (i) we first present the simulation setup, i.e., to specify the parameters passed as input to the HDFM simulator, (ii) we evaluate the performance degradation incurred when HDFM failing links are handled using classical methods, finally (iii) we test the performance of LR-HDD in detecting HDFM failing links and in the accuracy of the estimation of the lifetime.

### 6.1. Simulation setup

The performance of LR-HDD is tested over the Nobel network, available at [18], having 17 nodes and 52 links. In [18], also links capacities are reported. Furthermore, a set of real Traffic Matrices (TMs) related to a period of 24 hours are available. Each of them is referred to a time slot of 5 min, for a total of 288 TMs. Buffer length for each link ($b_l$) is selected so that the packet loss probability due to congestion is at most equal to $Q(b_l) = 10^{-2}$. We assume that all packets have the same length of $L_{\text{PKT}} = 1500$ Bytes.

The hardware deterioration functions considered in the simulations are those reported in Table 1, and are represented by straight lines. We recall that they are divided into 4 groups, each one having different characteristics. Group 1 is intended to model failures having short duration and limited time dependence, but leading to frequent disruptions (small $\delta_T$ and $\lambda$, but large $\phi$). This is the case of failure events on links that, during the creation of the data set (see Section 4), have been observed just before they enter in not operational status. We refer to models included in group 1 as *aggressive* failures. Models included in group 2 have a behavior similar to the ones of group 1, except for the duration. In particular, for these models the failing phase is 10 times longer than for hardware deterioration functions of group 1. This reflects the case of links that, during the creation of the data set, have been observed for a long period. Models of group 2 are also referred to as *hard* failures. Failures with a smoother behavior are included in group 3. They are characterized by an intermediate duration and a strong time dependence (large value of $\lambda$): at the beginning, the probability that the link is down is negligible, but becomes larger as the time since failure increases. These failures model links that, during the creation of the data set, have been observed since the first moment they entered in failing status. For this reason, we also refer to these models to as *wearing* failures. Finally, the main feature of failures modeled by hardware deterioration functions of group 4 is that they are time independent ($\lambda = 0$). These models are also referred to as *constant* failures, and are considered to test the performance of LR-HDD when failure events are not modeled as non stationary processes.

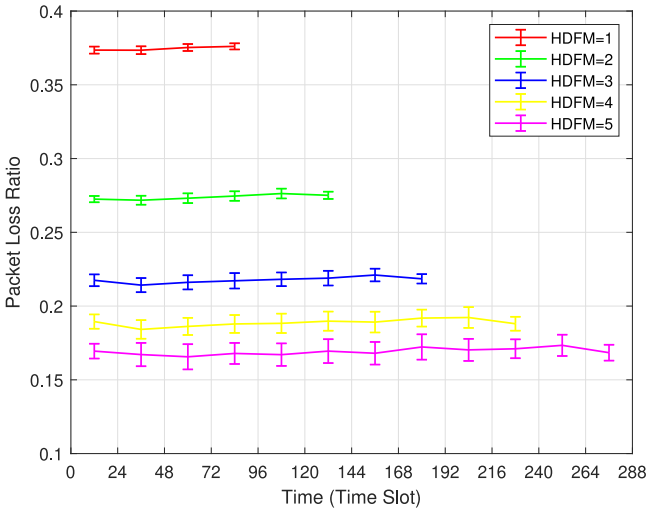



**Fig. 6.** Packet loss ratio of BFD protocol. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

### 6.2. Handling HDFM failing links with classical methods

As discussed in Section 3, classical methods, such as the use of an hello based protocol (e.g., BFD [19]) for the failure detection and the re-routing over an alternative path (e.g., FRR [20]) whenever the link is detected as down, is not a suitable way to handle failure events on links that are experiencing the HDFM. In the next we evaluate what is the negative impact of handling HDFM failing links with a classical approach, i.e., BFD+FRR.

The first analysis we propose aims at determining the percentage of packet loss incurred on a link experiencing the HDFM over time. For this test, *aggressive* failures are considered, since these are easier to be detected by BFD and their negative impact is incurred for a short period of time. BFD is configured with an *hello interval* of 1 s and a *dead interval* of 3 s. Results are averaged over all the network links and each failure is assumed to start at the beginning of each simulation (in the first time slot). The result of this analysis is reported in Fig. 6, having the time on the $x$-axis and the percentage of packet loss due to the failure on the $y$-axis. This last value accounts the packets that are transmitted on a link during the OFF[7] periods determined by the HDFM, while BFD

[7] In the simulation we assume that the link remains in OFF status for the duration of the transmission of a single packet.

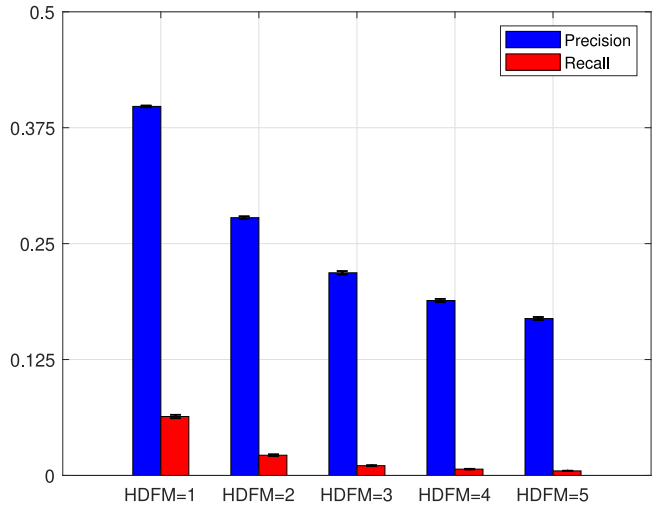



**Fig. 7.** Precision and recall metrics of BFD protocol.

detects the link as UP. In Fig. 6, lines have a different length due to the fact that each model has a different value of $\delta_T$. The main outcome of the performed analysis is that the packet loss is substantially high. This is a consequence of the fact that BFD, most of the time, does not detect the OFF periods of the failing link, that is consequently used to carry the traffic. In particular, BFD allows to get only a small reduction in the packet loss probability due to the HDFM dependent failures. As an example, considering the first model, it is characterized by an average packet loss probability of $\phi = 0.4$. Looking at Fig. 6 it can be seen that the use of BFD allows to get a decrease of this probability of about 0.025 (the average value for the red line is 0.375).

To better investigate the consequences of addressing HDFM related failures with a BFD+FRR approach, in the next we evaluate the precision and the recall of the detection phase. These two performance metrics are calculated according to Eqs. (5) and (6):

$$Precision = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}} \tag{5}$$

$$Recall = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}} \tag{6}$$

where the term TP represents the true positives, i.e., periods in which the link is in OFF status and it is correctly detected as DOWN, FP are the false positives, i.e., periods in which the link is in ON status but it is erroneously detected as DOWN, finally, FN represents the false negatives, meaning periods in which the link is in OFF status but it is erroneously detected as UP.

Precision and recall achieved by the BFD protocol in the detection of failures due to the HDFM are reported in Fig. 7. Looking at Fig. 7 it is evident that the precision of BFD is directly proportional to the value of $P_{\mathrm{OFF}}$. Clearly, in this condition it is more likely to lose hello messages and consequently detect the link as DOWN. Considerably, we have found that the best precision obtained by BFD is about 0.4, meaning that more than 60% of the re-routing operations are unneeded since these are performed for false positives, leading to a further QoS degradation.[8] Another interesting aspect regarding the precision obtained by BFD is concerned with the fact that, for each considered model, the precision tends to be equal to the $\phi$ parameter. This is due to the way we are modeling the transition between ON and OFF status for a link affected by the HDFM. In particular, for each transmitted packet, the link is OFF with a probability imposed by the hardware deterioration function, independently from what was its status during the transmission of the previous one (OFF events are independent). Consequently, during a period of time in which the link is detected as DOWN by BFD, the fraction of time in which it is actually OFF (determining a true positive) is, on average, $P_{\mathrm{OFF}}$.

As regard to the recall, a first interesting aspect to notice is that it has the same relation as the precision with the value of the $P_{\mathrm{OFF}}$. Also in this case, when $P_{\mathrm{OFF}}$ decreases the BFD detects a smaller number of status changes for the link, with a consequent reduction of the true positives with respect to false negatives. In particular, a smaller fraction of OFF periods of the link falls into the periods in which BFD detects the link as DOWN. Furthermore, it is worth mentioning that the value of the recall represents the reduction of the packet loss probability caused by the HDFM allowed by the use of BFD. As an example, we have seen how considering the model 1 of *aggressive* failures, the average OFF probability is $\phi = 0.4$, and that this value is reduced to 0.375 when BFD is used. Looking at the recall value reported in Fig. 7 for the model 1, it can be seen that its value is approximately 0.06, meaning that the 6% of the time that the link is OFF, BFD detects it as DOWN and the traffic is re-routed over an alternative path. Without BFD, the 40% of the time the failing link is used to carry traffic, despite it is in OFF status. This period is reduced of the 6% when BFD is considered, meaning that the percentage of time that the link is used in OFF status is $0.4 \cdot 0.06 = 0.375$.

To conclude, the conducted analysis has evidenced that handling HDFM related failures using classical methods leads to severe QoS degradation and has a negligible impact on mitigating the amount of packets lost, providing justification for the introduction of the proposed LR-HDD framework.

### 6.3. Evaluation of LR-HDD

The aim of this subsection is to evaluate the performance of the LR-HDD framework when it is used to handle failure events due to links affected by HDFM. Two different investigations are proposed: (i) evaluation of the accuracy of the estimation of the lifetime of failing links, and (ii) precision and recall in classifying links as failing or not failing. The first analysis aims at evaluating the remaining lifetime as a function of the time since failure. Once the lifetime of a line card is estimated, this value is used in the threshold based mechanism to assess if it is failing or not. Specifically, if the estimated lifetime is below the threshold, then the link is declared as failing. The aim of the second set of analyses is then to evaluate the performance of LR-HDD framework in assessing the link status. In particular, precision and recall as a function of the threshold are studied. In each figure, two lines are reported representing the actual lifetime and the one estimated through the LR-HDD.

In all the simulations, the data set is generated considering a single group of HDFM models reported in Table 1. Furthermore, the data set is divided into two parts, namely, *training* (used to train the ML algorithm) and *test* (used for evaluation) sets. For each considered group, we use the model reported in the third row of Table 1 for testing (since these are representative of the average behavior of the models belonging to each group), and all the remaining models (of the same group) for training.

Before commenting the obtained results, it is worth recalling the main idea on which the LR-HDD framework is based on. In particular, in order to provide an estimation of the lifetime of a link, LR-HDD tries to determine the packet loss probability due to the failure. Then, based on that, it tries to determine how much time it has passed since the failure has started, by exploiting the hypothesis that the failure is time dependent (non stationary process). Since this probability is not available in practice, i.e., there are no tools to measure it, LR-HDD uses four features to determine its value. By means of these four features, LR-HDD tries to distinguish between packets lost due to congestion events from the ones dropped due to the HDFM.

[8] We leave this aspect for evaluation in future work.

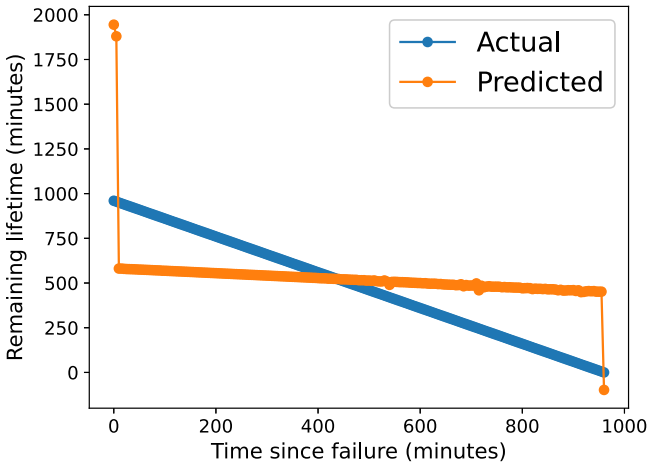



**Fig. 8.** Lifetime as a function of the time since failure for *aggressive* failures.

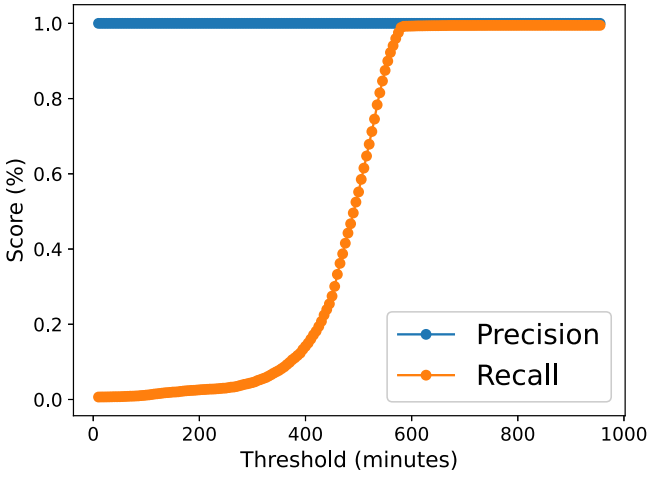



**Fig. 9.** Precision and recall of the threshold based mechanism fed with estimated lifetime for *aggressive* failures.

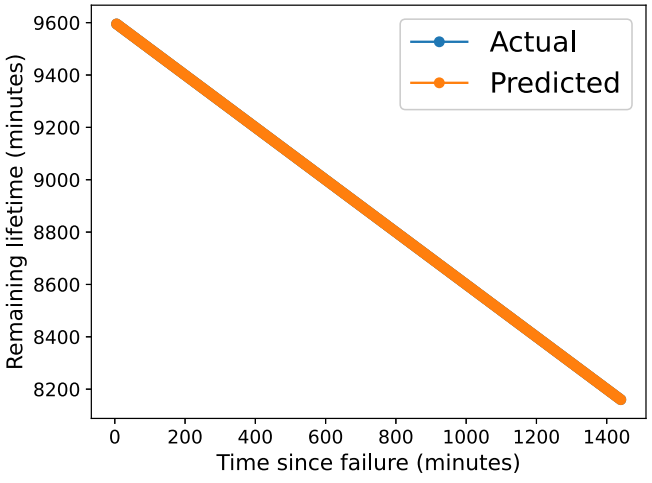



**Fig. 10.** Lifetime as a function of the time since failure for *hard* failures.

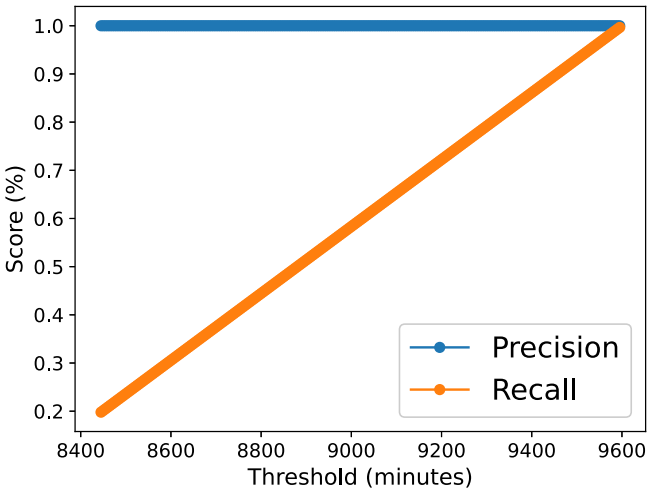



**Fig. 11.** Precision and recall of the threshold based mechanism fed with estimated lifetime for *hard* failures.

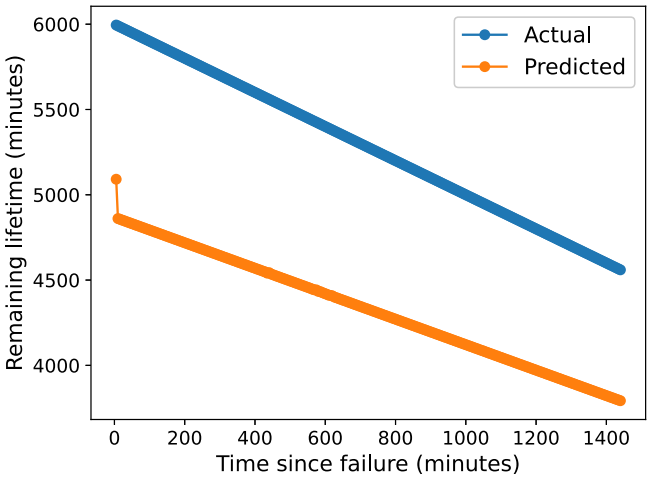



**Fig. 12.** Lifetime as a function of the time since failure for *wearing* failures.

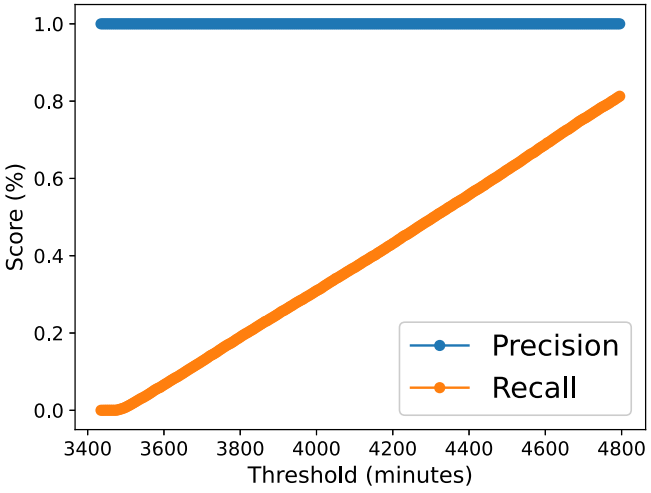



**Fig. 13.** Precision and recall of the threshold based mechanism fed with estimated lifetime for *wearing* failures.

In case of LR-HDD, the actual data set is a collection of observations having four features and a label. In the ideal case, the data set would be represented by observations having the packet loss probability due to the failure as single feature and the lifetime as label. Consequently, the ideal data set can be represented in a two dimensional space ($\mathbb{R}^2$), having the single feature as $x$-axis and the lifetime as $y$-axis. Despite the

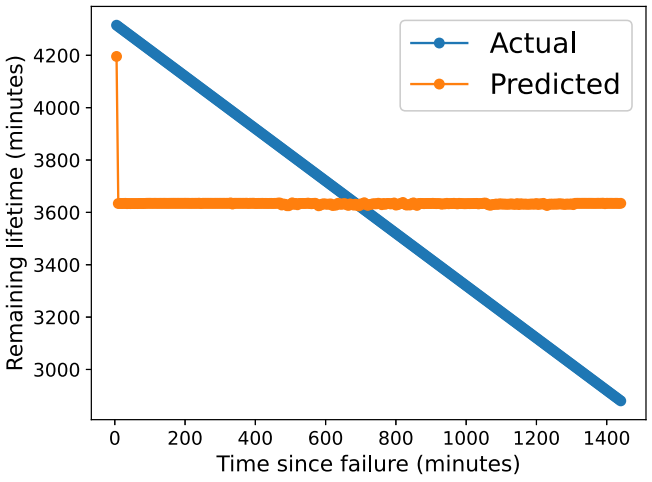



**Fig. 14.** Lifetime as a function of the time since failure for *constant* failures.

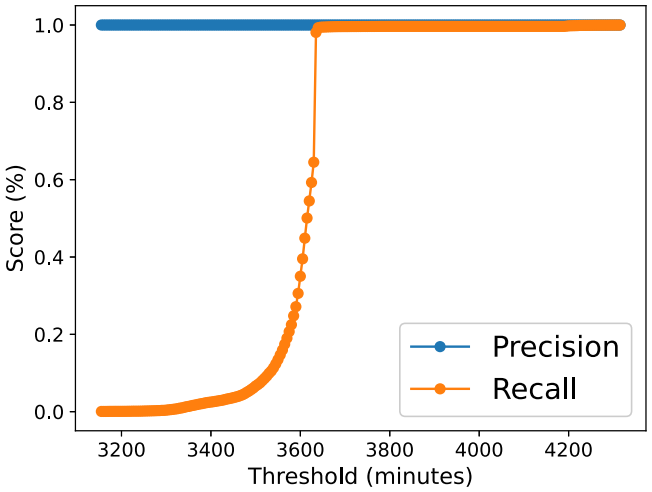



**Fig. 15.** Precision and recall of the threshold based mechanism fed with estimated lifetime for *constant* failures.

ideal data set is not available in practice, it provides an easier way to interpreter the results obtained over the actual data set, consequently in the next we refer to it to explain the outcomes of the proposed analyses. In particular, in the ideal data set it is possible to define the concept of *data shape*, that is a polyhedron in $\mathbb{R}^2$ containing all the observations. The type of polyhedron and its area (which is related to the density of the points) considerably affects the performance of LR-HDD. A further parameter that we introduce to comment the obtained results is the *Data Decrease Ratio* (DDR), which is defined as the slope of the straight line passing through the points (in $\mathbb{R}^2$) that are related to the following observations: (i) the observation having the highest value of lifetime and the smallest value of packet loss probability, and (ii) the observation with the lowest lifetime and the largest value of packet loss probability. The DDR can be calculated according to Eq. (7). and an example is reported in Fig. 16.

$$\text{DDR} = \frac{\delta_T^{\max}}{\phi^{\max} - \phi^{\min} + \lambda^{\max} + \lambda^{\min}} \tag{7}$$

In the next, the described tests are performed over each of the defined types of failure and the obtained results are commented. The first type of failure that is considered is the *aggressive* one. Results are reported in Figs. 8–9. A first outcome is that the LR-HDD in this case provides an almost flat estimation, i.e., the algorithm outputs an estimated lifetime that is independent from the time since failure.

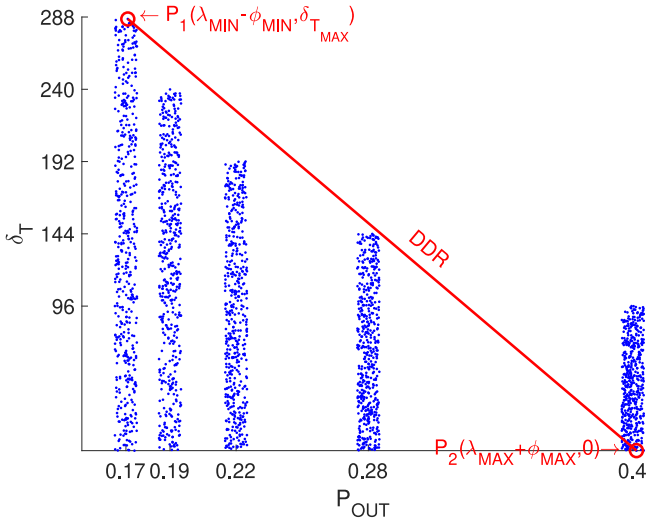



**Fig. 16.** Example of Data Decrease Ratio (DDR).

The reason for that is related to the nature of *aggressive* failures. In particular, these are characterized by a trapezoidal *data shape*, with the two bases along the $y$-axis with length 288 and 96 time slots respectively (the largest and smallest $\delta_T$ for *aggressive* failures), and height equal to 0.23 (the difference between the $\phi$ of the shortest and the longest *aggressive* failure). Furthermore, the DDR is about −1250, consequently the resulting regression line has a relatively small slope. These two facts imply that the final hypothesis has a limited dependence from the packet loss probability: small variations of the packet loss result in low changes in the estimated lifetime. This explains why the estimation produced by LR-HDD reported in Fig. 8 is almost independent from the time since failure. Regarding the precision and recall for *aggressive* failures, the results are shown in Fig. 9. It is interesting to notice that in both the considered cases the maximum precision is achieved, meaning that there are no false positives (non failing links are correctly classified). This is a crucial finding, since in the proposed methodology a link detected as failed must be replaced by the network operator. With reference to the recall parameter, it has a step behavior in case of LR-HDD. This is a consequence of the fact that the linear regression is providing as output an almost constant estimation for the lifetime. Consequently if the threshold is set above this value (around 600 time slots in the considered example), all the failing links are correctly detected, and there are not false negatives.

Figs. 10–11 show the results obtained in case of *hard* failures. Interestingly, in this case we have obtained the opposite behavior with respect to the case of *aggressive* failures. In particular, LR-HDD provides as output a really accurate estimation of the lifetime. The reason is that, also for *hard* failures the *data shape* is trapezoidal, having the same height, while the bases are 10 times longer than in case of models belonging to group 1. This implies that the DDR has an absolute value much larger than the one obtained in case of *aggressive* failures. In particular the DDR is about −12500. The consequence is that the regression line has a high slope, meaning that the final hypothesis is strongly dependent from the packet loss probability. Then, small variations of this parameter turn into significant variation on the estimated lifetime. Regarding the precision, also in this case non failing links are correctly detected, leading to 0 false positives. As a consequence, the maximum precision is always achieved. Concerning the recall, since in this case LR-HDD is better approximating the actual lifetime, links are not detected as failing as long as their actual lifetime becomes lower than the threshold. Here we emphasize that this behavior is more attractive than the one obtained for *aggressive* failures, since it allows the operator to decide for how long to use a failing link before substituting it.

The analyses related to *wearing* failures are reported in Figs. 12–13. Also in this case LR-HDD allows to get a quite good estimation for the lifetime. Regarding the *data shape*, it is represented by the portion of the space contained between the two straight lines related to the shortest and longest failures. In particular, with reference to the plane having the packet loss probability due to the failure on the $x$-axis and the lifetime on the $y$-axis, these straight lines pass through the points $(0, \delta_T)$ and $(\phi, 0)$. The DDR for this type of failure is about $-3600$. In this case, the regression line has a slope that is not significantly higher than the one obtained for *aggressive* failures. Despite this, the produced estimation has a good quality. This is due to the fact that *wearing* failures have a strong time dependence (large value of $\lambda$), consequently also if the final hypothesis has a limited dependence from the packet loss probability, since this is highly variable, the algorithm is still able to produce good estimations. Results related to the precision confirm once more that there are not false positives independently from the considered threshold. On the contrary, if we consider the recall, it can be seen that it never reaches the maximum value. This is due to the fact that the outage probability for *wearing* failures is null when the failure starts. Consequently, in $t = 0$, LR-HDD is not able to distinguish between a non failing line card and a link failing according to the *wearing* model.

Finally, results for the *constant* failures are presented in Figs. 14–15. We recall that, failures belonging to this group are not conformed to the hypothesis that is at the basis of the present work, i.e., that the hardware deterioration is a non stationary process. As expected, the estimation of the lifetime is constant for *constant* failures. Anyway, the reason for this finding is not due to the *data shape*, which also in this case is represented by a trapezoid, whose bases have a length of 288 and 1440 time slots respectively, while the height is 0.14. Consequently, the area of the *data shape* is bigger than for *aggressive* failures and lower than the one obtained for *hard* failures. This leads to a regression line having a significant slope. Despite this, the estimated lifetime is flat. This is due to the fact that even though the regression line is sensible to small variations of the packet loss probability, since the failure is constant, the final hypothesis is always fed with the same value. The consequence is that, when the estimation is used in conjunction with the threshold based mechanism, the recall has a step behavior (see Fig. 15). On the contrary, the precision is still equal to the maximum value independently from the value of the threshold.

Before concluding we summarize the main outcomes of the conducted analysis. As first, we have seen that for some types of failures (*aggressive* and *constant*) LR-HDD does not produce an accurate estimation for the lifetime. The reasons of this behavior are different: (i) while in case of *aggressive* failures the poor quality in the lifetime estimation is due to the low value of DDR exhibited by the data set, (ii) for *constant* failures the reason is that those models do not reflect the non stationary behavior of HDFM. Anyway, also for these models, LR-HDD is still able to correctly classify links as failing or not failing. Another important finding from the conducted analysis is related to the non stationary hypothesis exploited by LR-HDD to estimate the lifetime of failing links. As it has been shown, the best results are obtained for *hard* failures, having a very limited dependence on the time. This is an interesting outcome, since it allows to apply the methodology also in case of hardware that deteriorates slowly over time. The last remarkable outcome of the performed evaluation is that we have identified overall descriptions of failure models for which LR-HDD produces a high quality estimation of their lifetime. In particular, we have found that there is a trade-off between the time dependence of the hardware deterioration function ($\lambda$) and the DDR. High values of DDR allow to handle almost constant failures.

### 6.4. Sensitivity analysis

In this subsection we provide some insights on the proposed solution. In particular, three different aspects are covered. As first we provide an evaluation of the importance of each of the features of the data set in the decision process. Then, we show the performance achieved by considering Neural Network as the ML technique for HDD. Finally, a sensitivity analysis of HDD is performed over models that are trained with data sets referred to hardware deterioration functions different to the ones considered in the test.

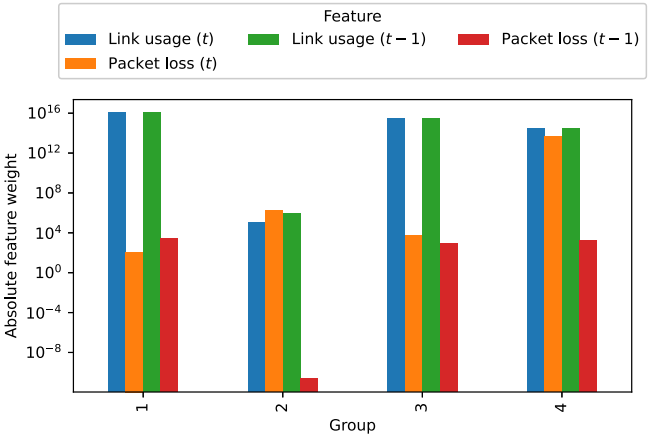



**Fig. 17.** Absolute weight (importance) of each of the features for the different groups.

The first analysis aims at evaluating the impact of each of the four features (link usage and packet loss at times $t$ and $t-1$) on the prediction reported by the framework. Fig. 17 reports the absolute weight of each feature for every group of failures: 1 - *aggressive*, 2 - *hard*, 3 - *wearing*, 4 - *constant*. By looking at Fig. 17, it can be seen that the most important features in the decision process are the ones related to link usage, both for the current time slot $t$ and for the previous one $t-1$, for all the considered types of failures. However, the importance of packet loss is not as high as in the case of link usage, especially when referring to the previous time slot (see, e.g., group 2). In fact, link usage is used to detect how many packets are lost due to congestion, while packet loss is exploited to estimate the $P_{OFF}$ probability. In this way, considering the four groups of failures, the number of features could be reduced from 4 to 3, removing from the set the one related to packet loss at previous time slot.

The next sensitivity analysis that we propose consists in comparing the quality of the estimated lifetime using a different ML technique, i.e., Neural Network (NN). The leveraged NN is a Multi-Layer Perceptron Regressor with an input layer of 4 neurons (one per feature), a hidden layer of 100 neurons and an output layer with a single neuron, all of them using a ReLU activation function. We tested all the failure modes reported in Table 1, but only in two of them LR and NN got different results, i.e., *aggressive* and *wearing* failures.

Results of this analysis are reported in Figs. 18 and 19. As it can be seen, in case of *aggressive* failures the use of NN allows for an improvement of the quality of the estimated lifetime. On the other hand, when *wearing* failures are considered, LR obtains better and more stable result than NN.

The hardware deterioration of real network devices is a phenomenon that is expected to last in the order of years. At the same time, networks are continuously evolving environments, with periodic technology upgrades. This leads to the creation of a data set that is referred to line cards differing in many aspects, such as technology, age, vendor type, deteriorated components, etc. This situation is equivalent to test HDD over a test set composed of observations generated by hardware deterioration functions that were not considered for the creation of the training set. Clearly, this fact can represent a limiting factor for the applicability of the proposed methodology.

In the next we present a sensitivity analysis to evaluate the impact of the previously discussed situation. The performed experiments consist

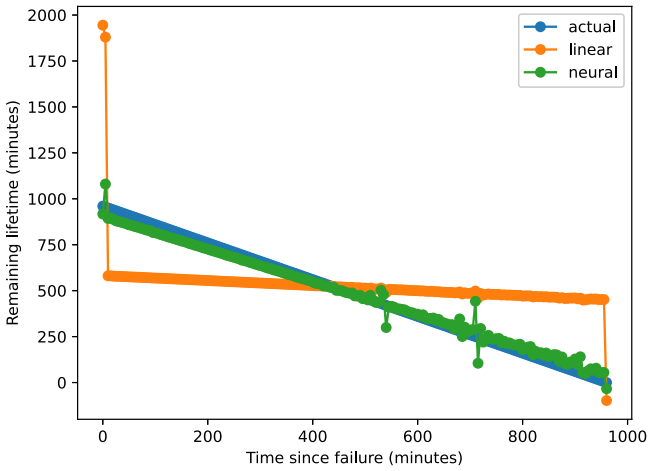



**Fig. 18.** Comparison between LR and NN in case of *aggressive* failures.

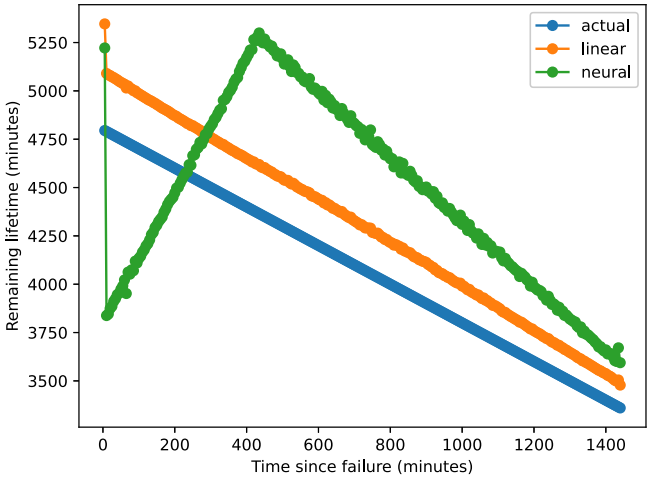



**Fig. 19.** Comparison between LR and NN in case of *wearing* failures.

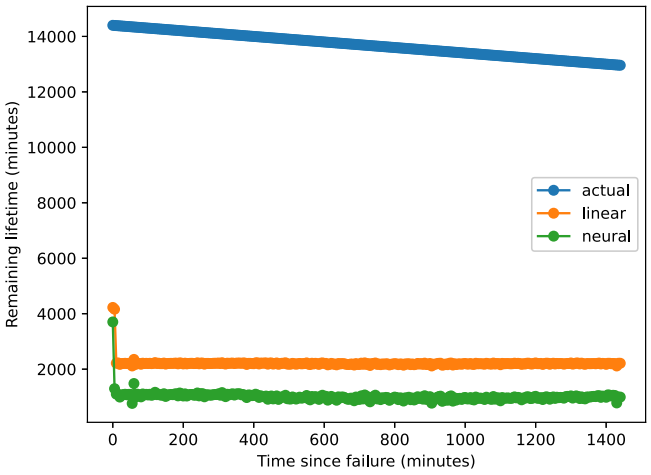



**Fig. 20.** Test performed on *hard* failures.

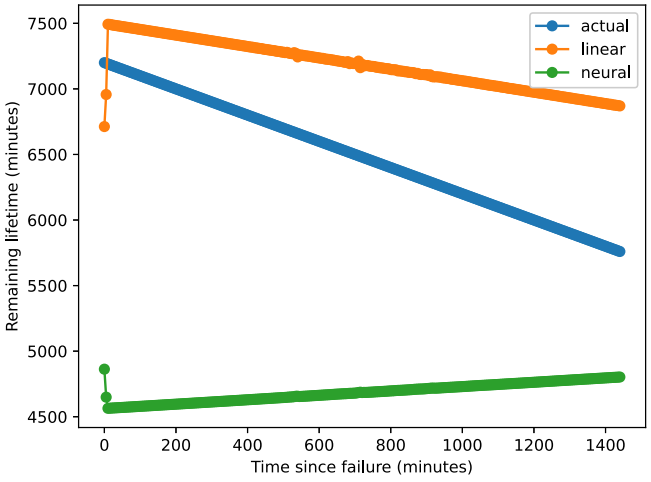



**Fig. 21.** Test performed on *wearing* failures.

in evaluating the performance of HDD when it is trained over a data set that includes observations referred to all the models presented in Table 1 except one, which is instead considered to generate the test set. Two different failure modes are considered for the test, i.e., *hard* and *wearing*. Results are shown in Figs. 20–21. Also in this case LR and NN are considered.

Regarding the result obtained in case of *hard* failure, it is clear from Fig. 20 that the estimation error is high. In particular the estimation reported in output is a pessimistic one, with an estimated lifetime that is about ten times lower than the actual one. The reason for that is the presence in the training set of the *aggressive* failures that from the $P_{OFF}$ perspective behave like the *aggressive* ones, but last ten times less. Despite this, the classification of line cards as failing or not failing is still working. In particular, due to the fact that the estimation is a pessimistic one, the estimated lifetime is always smaller of the actual one. Consequently whatever the value of the threshold is, if the actual lifetime is below the threshold, also the estimated one it is. Unfortunately the opposite does not hold. As a consequence, the failing line card will be substituted much earlier than expected (the selected threshold value). To conclude, in this case LR and NN get similar result.

A different situation is obtained in case of *wearing* failures. Looking at Fig. 21 it can be seen that the estimation error in this case is lower. The reason is that, differently from the previous case, in the training set there are no models with a behavior similar to *wearing* failures. Consequently, there is no bias in the estimation. Regarding the ability to correctly classify failing line cards, also in this case the method still works. As an example, let us consider that the threshold value is 7000 min that is reached in the actual situation after about 200 min since the failure has started. Looking at Fig. 21 the situation is as follows. In case of NN, the link is immediately detected as failing, then the considerations done in the case of *hard* model hold. In case of LR it is correctly classified after 1000 min since the failure started. It implies that the method is affected by a delay in the failure detection. Despite this, the failure is detected much earlier than the card enters in not operational status.

Before concluding, it is important to highlight that in the case reported in Fig. 21 it is clear that the estimation produced by LR slowly diverges from the actual line, while NN one converges. This fact suggest that the use of more powerful ML models can help HDD to work on data sets that contain observations related to hardware having heterogeneous technology.

## 7. Lessons learned

Despite the performance evaluation has been carried out over synthetic data, it has allowed to get important feedback on the proposed framework. For this reason, in this section we describe what we have learned from our experience.

The first encouraging lesson learned is that the detection of HDFM affected links is possible also in the pessimistic case of failures modeled

by a stationary process. This represents the worst case scenario for the proposed framework, since it exploits the time dependence of the outage probability to guess the current status of the link. Clearly in this situation the estimated lifetime has a poor quality, but the detection is still feasible.

Another important lesson learned is about the data collection process. As the performance evaluation has evidenced, the quality of the estimated lifetime is strongly related to the type of failure that is considered for training. In particular, we have found that *aggressive* failures are hard to manage, due to their nature (the card expires after relative few time with respect to the moment in which it is tested by the algorithm). On the contrary, long living failures, such as the *hard* and *wearing* ones, allow for high quality estimation of the lifetime. As pointed out on the performance evaluation, the difference among the data sets produced by these different models is in the DDR parameter, where the rule of thumb is that data sets with high DRR allows for the creation of a more sensible model. Referring to Eq. (7), there are two ways to get high values of DDR: (i) build a training set containing data referred to failing links having similar values of the outage probability, or (ii) create a data set including samples coming from failing links characterized by large difference in the failure duration. Unfortunately there is not a practical way to get an accurate knowledge of the outage probability. Thus a feasible way for the network operator to build a training set with a large DDR is to exploit the second observation, i.e., to consider failures having different duration. This translate in the following simple rule: get data over a large time window. In this way it is more likely to consider failures characterized by different duration.

In Section 4 we have described the structure of the considered observations, explaining that these includes four features, i.e., the link load in current and previous time slot plus the packet loss in current and previous time slot. These features represent the minimum set of information that allows to detect line cards in HDFM status and their lifetime. Obviously, there are many useful features that can help the framework in its goal, such as, advanced statistics on the traffic distribution, the line card model with its hardware characteristics, the line card age, historical information, etc. Unfortunately, due to the lack of such data, we have decided to consider the minimal set of features. Clearly, with a more larger set of features, it might be beneficial for the framework to consider more advanced ML models. In our experience we have seen that, with this limited data set, the fact of having flexible algorithm do not lead to performance improvements.

With this lessons learned, we provide network operators guidelines to improve the detection process of failing line cards in the network. With these guidelines, they count with more information to decide the action to be performed, depending if the failing line card is in an early or late HDFM stage.

## 8. Related work

In this section, a literature review of works dealing with the failure recovery problem in communication networks is presented. Furthermore, a set of works proposing ML based solutions for improving link failures recovery are reviewed.

### 8.1. Failure of electronic circuit boards

An interesting overview of the existing methods for Circuit Board Failure Detection and Prediction is reported in [8]. Six broad method categories are defined and presented. The first one is the *periodic inspection*, which aims at detecting aging in circuit board components assuming that the aging condition produces an observable measure during the test (e.g., as an increase in the time for circuit actuation), or in the case of visual inspection a color change somewhere on the printed circuit board. Methods falling into the second category are based on the definition of *reliability models*, which assume that statistical evaluations of the failures of components in a large population under accelerated aging conditions can be used to generate failure rates that can be applied to similar components in other applications. The third category includes methods that exploit the concept of *resistance measures*. The idea is that, over time, slow chemical reactions within the insulating material lead to lower resistance pathways between the conductor and the ground. This change is measurable through simple resistance measures or by noting the increase in leakage current. Methods belonging to the fourth category are based on the *theory for signal comparison*. The idea is that if two measures are supposed to be the same and actually they are not, then something has changed such as aging of a circuit component. *Passive measurement systems* belong to the fifth category, where measurements target either the circuit (e.g., the continuous measure of the current flow) or the environment (such as temperature, vibration, or air quality). The last category is referred to as *active measurement systems*. These methods are based on the perturbation theory, and try to detect the aging effects in the circuit as of changes in response measurements to signal perturbations over time.

In the scenario we consider, i.e., the one of failures on network line cards due to the hardware deterioration, techniques such as periodic inspection, resistance measures and the ones based on the theory of signal comparison are not practically suitable, since they require a direct interaction with the hardware, while networks are generally geographically spread infrastructures (remote control is preferred). Reliability models have been applied on network equipment to estimate the mean time between failures. An example is reported in [21], where the impact of ON/OFF strategies to reduce the power consumption of the network infrastructure is evaluated through a reliability model. Anyway, this type of approaches require very detailed information on the individual components and parameters (such as materials, type of package, model, technology, application, etc.) [22], which in general are difficult to know.

With respect to the aforementioned classification, the method proposed in this paper can be seen as a *passive measurement system*. Anyway, instead of measuring currents and voltages, it focuses on parameters that describe the way the system is working and, based on them, tries to detect possible anomalies in the normal behavior due to hardware deterioration. As it will be show in the result section, *active measurement systems* are not suitable in case of network line cards, leading to poor precision and recall in assessing the aging condition of the hardware.

### 8.2. Handling link failures in communication networks

Link failures have emerged as the most prominent type of failure that can be found in a computer network [4]. A fast detection of a failure in a link is critical for routing purposes in order to maintain acceptable QoS levels [23]. Generally, the control plane is responsible of acting after the detection of a link failure with the computation of alternative paths to steer the traffic. Routing schemes such as OSPF and IS-IS include mechanisms based on the exchange of global messages to detect when a link is failing and to act accordingly. FRR mechanisms are widely deployed on current networks to leverage precomputed alternative paths for every source–destination pair. With the application of this type of solutions, a decrease in the performance degradation of the network is experienced. Different solutions for FRR are applied depending on the underlying technology: IP FRR for IP networks [20], MPLS FRR in case of MPLS networks [24], OpenFlow fast-failover groups for SDN [25], or BGP-PIC in case of BGP protocol connecting different Autonomous Systems [26].

However, the rerouting techniques introduced above are composed of a set of steps: *failure detection*, *transitory period* and *recovery notification*. Regarding the *failure detection*, a failure can be detected by analyzing the potential variation in different parameters, such as Bit Error Rate (BER), CRC computation, or Checksum corroboration, whose deviation is associated to the existence of an abnormal behavior in the

communication between devices. End-to-end QoS parameters decrease, such as a lower throughput or an increased delay or jitter, is another way for failure detection [27].

In case of multi-layer networks (e.g., IP-over-WDM architectures), two types of schemes are used for failure detection: *active* and *passive* schemes [28]. An active scheme relies on the sending of periodic keep-alive messages, such as the Bidirectional Forwarding Detection (BFD) protocol [19], while passive approaches only consider data packets that are flowing throughout the network in order to detect potential anomalies in the networking equipment [29]. While the former set of approaches necessarily increase the network traffic due to control messages, the latter has the drawback of not being accurate in low traffic situations.

In Software-Defined Networks (SDN) networks, the decoupling of the data plane from the control plane and the key impact of the logically centralized controller add complexity to link failures management. If a link failure occurs, this situation must be first detected and after that notified to the controller (or controllers in case of multi-domain SDN), which is in charge of re-configuring the affected paths. An increase in the experienced delay, as well as the possibility of not convergence in case of in-band signaling, are challenges that must be tackled when handling failures in SDN-based scenarios [30–32]. In fact, in a carrier-grade network it is required to achieve a sub-50 ms failure recovery [33]. This implies that the failure should be detected and the traffic should be restored prior to 50 ms. However, authors such as Van Adrichem et al. argue that the Ethernet nature of OpenFlow networks (and the relatively infrequent heartbeats) restrains the achievement of a high availability and fast failure detection (below 50 ms) [32]. In fact, the experiments of [34] report that this value is exceeded (on the order of 100 ms) in large-scale carrier grade networks due to the increase of the controller reaction times. A faster alternative could be to combine the use of BFD [19] with primary and backup paths configured by the SDN controller. Another option to reduce the recovery time is to exploit FRR mechanisms in MPLS networks along with the global network view of an SDN controller [30].

After the detection of a failure, a *transitory time* is started in which a degradation in the network QoS is experienced (e.g., packet loss or delay increase, throughput decrease, etc.). During the *recovery notification* phase, flows that are affected by the detected failure are redirected to their respective alternate paths, which can be computed following two different approaches: (i) *offline pre-computation*, which aims at finding a suitable routing (and protection) scheme for a given traffic matrix that is predicted based on historical data [34,35]; and (ii) *online reconfiguration*, in which alternative paths are online computed for rerouting the flows in case of failures [36]. In the latter option, protection schemes are used to exclude the failing link from the potential candidate set in the future. Since network operators pursue the lowest service recovery time as well as the highest Quality of Resilience (QoR), offline pre-computation schemes (with backup paths set before the failure) are commonly used, redirecting the traffic over a small (and close) portion of the network. These non-optimal paths are maintained during the time period while the faulty element (node, link) is being repaired. After that, a normalization phase is performed to revert the traffic to the original path that was used before the failure.

In order to improve the network performance under failing situations, the networking research community has started to consider the possibility of predicting failures before they actually exist. The emergence of ML techniques applied to networking derived in a broad set of works that are described in the following section.

### 8.3. ML-based approaches to enhance link failure recovery

Concerning network failures, several ML-based solutions have addressed failure prediction to be applied to the field of network maintenance and operation. The application of ML in optical networks has attracted the attention of academia due to their inherent challenges: (i) a huge volume of data needs to be monitored in modern optical networks; (ii) different types of failures (fiber cuts, misconfiguration, equipment malfunctioning and aging, etc.); (iii) their inherent cross-layer nature (physical and network layers must be taken into account).

In [37], Musumeci et al. review the set of works that apply ML techniques to manage failures in optical networks, categorize the different phases that are inherent to failure malfunctioning and provide a description of how to practically solve a possible problem due to failures in this type of networks.

An early detection of soft-failures (e.g., signal degradation) [38] before a critical threshold is overcome would allow the operator to speed up the reaction and decrease the potential negative impact on the optical network. In [39], a service restoration strategy is proposed to proactively relocate cloud services that are traversing links which are predicted to fail. Using this proactive solution, a significant reduction in the number of cloud services that must be relocated is achieved.

The high prediction accuracy of the use of Long Short-Term Memory (LSTM) for failure prediction on power systems and satellite communication systems [40,41], opened the niche to exploit its use in optical networks. In particular, Zhang et al. propose in [42] a solution in which alarm analysis and failure prediction are combined to predict device failures at optical layer using LSTM and Support Vector Machine (SVM). Similarly, in [43] an adaptive approach based on LSTM which avoids the use of Double-Exponential Smoothing (DES) on assisting SVM for physical indication is proposed.

Concerning rerouting strategies based on predictions, patent [44] includes a traffic impact prediction module inside a network device to forecast topology-changing faults based on the current operating characteristics of the device. A probability on the traffic loss associated to the potential fault is assessed and the corresponding adjusted routing metric is performed. In [45], a controller device gathers flow information from the network elements to generate a prediction of a metric of source routing behavior within the network. A similar approach is described in [46], where network metrics are collected and a ML-based solution is proposed to predict failures that are likely to occur so that traffic is rerouted before the network element is actually failing.

In a previous work [6], the authors of the present paper proposed a ML-based approach derived from a linear regression to learn how to mimic the mathematical model behind the hardware deterioration process of failing line cards, and thus to estimate their remaining lifetime.

## 9. Conclusion and future work

In this paper we have investigated the hardware deterioration process of network line cards and its relationship with the packet loss probability. A characterization of link failures, divided into LFLs and HFLs is provided, for which an early detection can help the operator to take actions in advance on the routing strategies (LFLs) or on links replacement (HFLs) to reduce the potential impact on the QoS degradation.

For this reason, a strategy to manage failure events due to the hardware deterioration of HFLs is proposed. In particular, a ML-based solution, HDD, is provided to detect if a link is actually a HFL and to assess its remaining lifetime. With this information, the operator is able to decide the action to be performed: (i) to replace the failing link and the time instant at which it must be replaced (e.g., an old line card), or (ii) to accept the QoS degradation derived by its usage (e.g., an infant line card), compared with the cost of a link replacement. The definition and formalization of the HDFM mode for failing links is also provided, describing the behavior of a HFL affected by the hardware deterioration process, which is assumed to be non stationary. A network packet based failure simulator has been developed to model HDFM on line cards and to generate the data set used to evaluate HDD. As first, we proved that

handling HDFM related failures using classical methods (BFD+FRR) leads to severe QoS degradation, with low values for precision and recall. After evaluating the proposed HDD solution taking into account different types of failures (characterized as *aggressive*, *hard*, *wearing* and *constant*), the main outcomes that have been found are that HDD is able to correctly classify links as failing or not failing, regardless the type of considered failure. Although the estimation of the remaining lifetime reported by HDD on links affected by *aggressive* and *constant* HDFM is not as accurate as in case of *hard* and *wearing*, an overall description for failure models in which HDD is able to produce high quality estimations of the remaining lifetime has also been provided, with a trade-off between the hardware deterioration time dependence and the relationship between lifetime and packet loss probability (DRR).

The proposed framework has also highlighted two future research directions. The first one is the creation of an event-based network simulator for network failures in order to feed HDD with a thinner data granularity. Another research activity can be the application of HDD to real data derived from real networking equipment or network emulators such as Mininet.

## CRediT authorship contribution statement

**Marco Polverini:** Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Resources, Data curation, Writing - original draft, Writing - review & editing, Visualization, Supervision, Project administration. **Juan Luis Herrera:** Conceptualization, Software, Validation, Formal analysis, Investigation, Resources, Data curation, Writing - original draft, Writing - review & editing, Visualization. **Pierpaolo Salvo:** Conceptualization, Software, Validation, Formal analysis, Investigation, Resources, Data curation, Writing - original draft, Writing - review & editing, Visualization. **Jaime Galán-Jiménez:** Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Resources, Data curation, Writing - original draft, Writing - review & editing, Visualization, Supervision, Project administration, Funding acquisition.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Acknowledgments

This work has been partially funded by the project RTI2018-094591-B-I00 (MCI/AEI/FEDER,UE), the 4IE+ Project (0499-4IE-PLUS-4-E) funded by the Interreg V-A España-Portugal (POCTEP) 2014–2020 program, and by the Department of Economy, Science and Digital Agenda of the Government of Extremadura, Spain (GR18112, IB18030).

## References

[1] A. Markopoulou, G. Iannaccone, S. Bhattacharyya, C.N. Chuah, Y. Ganjali, C. Diot, Characterization of failures in an operational IP backbone network, IEEE/ACM Trans. Netw. 16 (4) (2008) 749–762, http://dx.doi.org/10.1109/TNET.2007.902727.

[2] M.A. Lema, A. Laya, T. Mahmoodi, M. Cuevas, J. Sachs, J. Markendahl, M. Dohler, Business case and technology analysis for 5g low latency applications, IEEE Access 5 (2017) 5917–5935.

[3] D. Tweney, 5-minute outage costs Google $545,000 in revenue, 2021, Available at URL https://venturebeat.com/2013/08/16/3-minute-outage-costs-google-545000-in-revenue/ (last accessed on 15th March 2021).

[4] C. Labovitz, G.R. Malan, F. Jahanian, Internet routing instability, IEEE/ACM Trans. Netw. 6 (5) (1998) 515–528, http://dx.doi.org/10.1109/90.731185.

[5] Q. Li, C. Wang, B.R. Ellingwood, Time-dependent reliability of aging structures in the presence of non-stationary loads and degradation, Struct. Saf. 52 (2015) 132–141.

[6] J.L. Herrera, M. Polverini, J. Galán-Jiménez, A machine learning-based framework to estimate the lifetime of network line cards, in: NOMS 2020-2020 IEEE/IFIP network operations and management symposium, IEEE, 2020, pp. 1–5.

[7] D. Medhi, K. Ramasamy, Chapter 13 - router architectures, in: D. Medhi, K. Ramasamy (Eds.), Network Routing (Second Edition), second ed., in: The Morgan Kaufmann Series in Networking, Morgan Kaufmann, Boston, 2018, pp. 424–452, http://dx.doi.org/10.1016/B978-0-12-800737-2.00016-8, URL https://www.sciencedirect.com/science/article/pii/B9780128007372000168.

[8] G.W. Hannaman, C.D. Wilkinson, Evaluating the Effects of Aging on Electronic Instrument and Control Circuit Boards and Components in Nuclear Power Plants, Tech. Rep., Electric Power Research Institute (US), 2005.

[9] D. Turner, K. Levchenko, A.C. Snoeren, S. Savage, California fault lines: understanding the causes and impact of network failures, in: Proceedings of the ACM SIGCOMM 2010 Conference, 2010, pp. 315–326.

[10] H. Qi, S. Ganesan, M. Pecht, No-fault-found and intermittent failures in electronic products, Microelectron. Reliab. 48 (5) (2008) 663–674, http://dx.doi.org/10.1016/j.microrel.2008.02.003, URL http://www.sciencedirect.com/science/article/pii/S0026271408000474.

[11] OSI IS-IS Intra-domain Routing Protocol, RFC 1142, 1990, http://dx.doi.org/10.17487/RFC1142, Available at https://rfc-editor.org/rfc/rfc1142.txt (last accessed on 15th March 2021).

[12] Cisco Systems, Enhanced interior gateway routing protocol, 2019, [Online]: https://www.cisco.com/c/en/us/support/docs/ip/enhanced-interior-gateway-routing-protocol-eigrp/16406-eigrp-toc.html.

[13] A. Kamisiński, Evolution of ip fast-reroute strategies, in: 2018 10th International Workshop on Resilient Networks Design and Modeling (RNDM), 2018, pp. 1–6.

[14] J. Galán-Jiménez, M. Polverini, A. Cianfrani, A scalable and error-tolerant solution for traffic matrix assessment in hybrid ip/sdn networks, IEEE Trans. Netw. Serv. Manag. 17 (1) (2020) 251–264.

[15] I. Goodfellow, Y. Bengio, A. Courville, Deep Learning, MIT Press, 2016, http://www.deeplearningbook.org.

[16] R. Fletcher, Practical Methods of Optimization, Wiley, 2013.

[17] I. Norros, J.W. Roberts, A. Simonian, J.T. Virtamo, The superposition of variable bit rate sources in an atm multiplexer, IEEE J. Sel. Areas Commun. 9 (3) (1991) 378–387.

[18] S. Orlowski, M. Pióro, A. Tomaszewski, R. Wessäly, SNDlib 1.0–survivable network design library, in: Proceedings of the 3rd International Network Optimization Conference (INOC 2007), Spa, Belgium, 2007.

[19] D. Katz, D. Ward, Bidirectional Forwarding Detection (BFD), RFC 5880, 2010, http://dx.doi.org/10.17487/RFC5880, Available at https://rfc-editor.org/rfc/rfc5880.txt (last accessed on 15th March 2021).

[20] A. Jarry, Fast reroute paths algorithms, Telecommun. Syst. 52 (2) (2013) 881–888, http://dx.doi.org/10.1007/s11235-011-9582-5.

[21] L. Chiaraviglio, L. Amorosi, P. Dell'Olmo, W. Liu, J.A. Gutierrez, A. Cianfrani, M. Polverini, E. Le Rouzic, M. Listanti, Lifetime-aware ISP networks: Optimal formulation and solutions, IEEE/ACM Trans. Netw. 25 (3) (2017) 1924–1937, http://dx.doi.org/10.1109/TNET.2017.2665782.

[22] M. Conti, S. Orcioni, Modeling of failure probability for reliability and component reuse of electric and electronic equipment, Energies 13 (11) (2020) 2843.

[23] S. Iyer, S. Bhattacharyya, N. Taft, C. Diot, An approach to alleviate link overload as observed on an IP backbone, in: Proceedings IEEE INFOCOM 2003, the 22nd Annual Joint Conference of the IEEE Computer and Communications Societies, San Francisco, CA, USA, March 30 - April 3, 2003, 2003, pp. 406–416, http://dx.doi.org/10.1109/INFCOM.2003.1208692.

[24] A. Atlas, G. Swallow, P. Pan, Fast Reroute Extensions to RSVP-TE for LSP Tunnels, RFC 4090, 2005, http://dx.doi.org/10.17487/RFC4090.

[25] A. Nygren, B. Pfaff, B. Lantz, B. Heller, C. Barker, C. Beckmann, D. Cohn, D. Malek, D. Talayco, D. Erickson, et al., Openflow Switch Specification Version 1.5. 1, Tech. Rep, Open Networking Foundation, 2015.

[26] Configuring BGP PIC Edge and Core for IP and MPLS, Tech. rep., Cisco., 2017, Available at https://www.cisco.com/c/en/us/td/docs/ios-xml/ios/iproute_bgp/configuration/xe-3s/irg-xe-3s-book/irg-bgp-mp-pic.html (last accessed on 15th March 2021).

[27] P. Cholda, A. Jajszczyk, Recovery and its quality in multilayer networks, J. Lightwave Technol. 28 (4) (2010) 372–389.

[28] S. Zhuang, D. Geels, I. Stoica, R.H. Katz, On failure detection algorithms in overlay networks, in: Proceedings IEEE 24th Annual Joint Conference of the IEEE Computer and Communications Societies, Vol. 3, 2005, pp. 2112–2123, vol. 3.

[29] T. Holterbach, E.C. Molero, M. Apostolaki, A. Dainotti, S. Vissicchio, L. Vanbever, Blink: Fast connectivity recovery entirely in the data plane, in: Proceedings of the 16th USENIX Conference on Networked Systems Design and Implementation, in: NSDI'19, USA, 2019, pp. 161–176.

[30] M. Chiesa, A. Kamisiński, J. Rak, G. Rétvári, S. Schmid, A Survey of Fast Recovery Mechanisms in the Data Plane, TechRxiv, 2020, http://dx.doi.org/10.36227/techrxiv.12367508.v2, URL https://www.techrxiv.org/articles/preprint/Fast_Recovery_Mechanisms_in_the_Data_Plane/12367508/2.

[31] A.S. da Silva, P. Smith, A. Mauthe, A. Schaeffer-Filho, Resilience support in software-defined networking: A survey, Comput. Netw. 92 (2015) 189–207, http://dx.doi.org/10.1016/j.comnet.2015.09.012.

[32] N.M. Van Adrichem, B.J. Van Asten, F.A. Kuipers, Fast recovery in software-defined networks, in: 2014 Third European Workshop on Software Defined Networks, 2014, pp. 61–66, http://dx.doi.org/10.1109/EWSDN.2014.13.
[33] D. Brungard, M. Betts, S. Ueno, B. Niven-Jenkins, N. Sprecher, Requirements of an MPLS Transport Profile, RFC 5654, 2009, http://dx.doi.org/10.17487/RFC5654, URL https://rfc-editor.org/rfc/rfc5654.txt.
[34] S. Sharma, D. Staessens, D. Colle, M. Pickavet, P. Demeester, Openflow: Meeting carrier-grade recovery requirements, Comput. Commun. 36 (6) (2013) 656–665, http://dx.doi.org/10.1016/j.comcom.2012.09.011.
[35] S. Petale, J. Thangaraj, Link failure recovery mechanism in software defined networks, IEEE J. Sel. Areas Commun. (2020) 1.
[36] V. Padma, P. Yogesh, Proactive failure recovery in openflow based software defined networks, in: 2015 3rd International Conference on Signal Processing, Communication and Networking (ICSCN), 2015, pp. 1–6.
[37] F. Musumeci, C. Rottondi, G. Corani, S. Shahkarami, F. Cugini, M. Tornatore, A tutorial on machine learning for failure management in optical networks, J. Lightwave Technol. 37 (16) (2019) 4125–4139.
[38] S. Varughese, D. Lippiatt, T. Richter, S. Tibuleac, S.E. Ralph, Identification of soft failures in optical links using low complexity anomaly detection, in: 2019 Optical Fiber Communications Conference and Exhibition (OFC), 2019, pp. 1–3.
[39] C. Natalino, F. Coelho, G. Lacerda, A. Braga, L. Wosinska, P. Monti, A proactive restoration strategy for optical cloud networks based on failure predictions, in: 2018 20th International Conference on Transparent Optical Networks (ICTON), 2018, pp. 1–5.
[40] S. Zhang, Y. Wang, M. Liu, Z. Bao, Data-based line trip fault prediction in power systems using lstm networks and svm, IEEE Access 6 (2018) 7675–7686.
[41] T. Liu, K. Kang, H. Sun, Fault prediction for satellite communication equipment based on deep neural network, in: 2018 International Conference on Virtual Reality and Intelligent Systems (ICVRIS), 2018, pp. 176–178.
[42] M. Zhang, D. Wang, Machine learning based alarm analysis and failure forecast in optical networks, in: 2019 24th OptoElectronics and Communications Conference (OECC) and 2019 International Conference on Photonics in Switching and Computing (PSC), 2019, pp. 1–3.
[43] C. Zhang, M. Wang, M. Zhang, D. Wang, C. Song, L. Guan, Z. Liu, Adaptive failure prediction using long short-term memory in optical network, in: 2019 24th OptoElectronics and Communications Conference (OECC) and 2019 International Conference on Photonics in Switching and Computing (PSC), 2019, pp. 1–3.
[44] A. Yadav, Policy-based selective traffic reroute based on predicted traffic loss, 2017.
[45] S. Jeuk, G. Salgueiro, Optimizing source routing using machine learning, 2019.
[46] J.-P. Vasseur, G. Mermoud, S. Dasgupta, Triggering reroutes using early learning machine-based prediction of failures, 2017.



**Marco Polverini** received the Master's Degree in Telecommunications Engineering and the Ph.D. Degree in Information and Communication Engineering from the University of Rome La Sapienza, in 2010 and 2014, respectively, where he is currently a Research Fellow at the Department of Information, Electronic and Telecommunications Engineering. His main research interests are routing protocols for energy saving in IP networks, network traffic monitoring and measurement in next generation routing technologies.



**Juan Luis Herrera** received the Bachelor degree in software engineering from the University of Extremadura in 2019. He is a researcher at University of Extremadura's Computer Science and Communications Engineering Department. His main research interests include IoT, fog computing and SDN.



**Pierpaolo Salvo** received the Ph.D. in Information and Communication Engineering in 2014 and M.Sc. in Telecommunication Engineering in 2010, full marks (110/110) and summa cum laude, both from University of Rome "Sapienza". In 2010, he was a junior researcher at the CNIT. From 2011 to 2017, he held the positions of researcher at the "Department of Information, Electronic and Telecommunication Engineering" (DIET) of University of Rome "Sapienza". He is author of several IEEE conference and journal papers and serves in the evaluation commission of various ICT journals and conferences. He has contributed to various national and international research projects. Since 2018 he has been working at "Fondazione Ugo Bordoni" as ICT expert. His main research topics focus on 5G networks, Wireless Networks, Software Networking, Intelligent Transport Systems, Vehicular Networks and Cognitive Radio Networks.



**Jaime Galán-Jiménez** received his Ph.D. in Computer Science and Communications from the University of Extremadura (Spain) in 2014. He is currently working at the Computer Science and Communications Engineering Department, University of Extremadura, as an assistant professor. During the past years, he has spent several research and teaching periods at University of Rome Tor Vergata, and at University of Rome La Sapienza, Italy. His main research interests are Software-Defined Networks, Network Function Virtualization, UAV-based 5G networks planning and design, 5G provisioning in rural and low-income areas and Mobile Ad-Hoc Networks.